of the 42 curriculum by heal-ans

FTPRINTF

Description

Subject

Project anme	**libftprintf.a**
Turn in files	`Makefile, *.h, */*.h, *.c, */*.c`
Makefile	`NAME, all, clean, fclean, re`
External functs.	`malloc, free, write, va_start, va_arg, var_copy, va_end`
Libft authorized	Yes
Description	write a libarary that contains `ft_printf`, a function that will mimic the original `printf()`

- The prototype of `ft_printf` is; `int ft_printf(const char *, ...);`
- Here are the requirements:
  - Do not implement the buffer management of the original `printf()`.
  - Your founction has to handle the following conversiond: `%c`, `%s`, `%p`, `%d`, `%i`, `%u`, `%x`, `%X`
  - Your functoin will be compared against the original `printf`
  - You must use the command ar to create your library Using the libtool command is forbidden.
  - Your libftprintf.a has to be created at the root of your repository
  - Your header file must be named ft_printf.h and must contain the prototype of your ft_printf() function.
- You have to implement the following conversions:
  - `%c` Prints a single character.
  - `%s` Prints a string (as defined by the common C convention).
  - `%p` The void * pointer argument has to be printed in hexadecimal format.
  - `%d` Prints a decimal (base 10) number.
  - `%i` Prints an integer in base 10.
  - `%u` Prints an unsigned decimal (base 10) number.
  - `%x` Prints a number in hexadecimal (base 16) lowercase format.
  - `%X` Prints a number in hexadecimal (base 16) uppercase format.

- `%%` Prints a percent sign.

Step 1: toward understanding `variadic function`

Research questions

```
1. What is a variadic function in C?
2. What does ... mean in a function declaration?
3. What is va_list ?
4. What do va_start , va_arg , va_copy , and va_end do?
5. What are the default argument promotions?
6. Why are char and short promoted to int when passed through ... ?
7. Why should you not assume that va_list can be copied using ordinary assignment?
8. What happens if va_arg requests a type that does not match the actual argument type?
```

- Types
  - va_list holds the information needed by `va_start`, `va_arg`, `va_end`, `and va_copy`
- Macros
  - va_start enables access to variadic function arguments
  - va_arg accesses the next variadic function argument
  - va_copy makes a copy of the variadic function arguments
  - va_end ends traversal of the variadic function arguments

Argument Access Macros [^5]

```
- The type va_list is used for argument pointer variables
- void va_start (va_list ap, last-required)
- type va_arg (va_list ap, type)
- void va_end (va_list ap)
- void va_copy (va_list dest, va_list src)
```

> Example [^1]

```c
#include <stdarg.h>
#include <stdio.h>

void simple_printf(const char* fmt, ...)
{
    va_list args;

    for (va_start(args, fmt); *fmt != '\0'; ++fmt)
```

```
    {
        switch(*fmt)
        {
            case 'd':
            {
                int i = va_arg(args, int);
                printf("%d\n", i);
                break;
            }
            case 'c':
            {
                // A 'char' variable will be promoted to 'int'
                // A character literal in C is already 'int' by itself
                int c = va_arg(args, int);
                printf("%c\n", c);
                break;
            }
            case 'f':
            {
                double d = va_arg(args, double);
                printf("%f\n", d);
                break;
            }
            default
                puts("Unknown formatter!");
                goto END;
        }
    }
END:
    va_end(args);
}

int main(void)
{
    simple_printf("dcff", 3, 'a', 1.969, 42.5);
}
```

> Output

```
3
a
```

```
1.969000
42.500000
```

Step 2: Understanding the original `printf` function [^2]

Research questions

```
1 What is a format string?
2 How does printf interpret conversion specifiers?
3 What is the return value of printf ?
4 How are null string pointers handled on your target system?
5 What is the difference between signed and unsigned conversions?
6 How are integers represented in hexadecimal?
7 How does %p display a pointer?
8 What happens when the format string contains an invalid or incomplete conversion?
```

Conversion specifiers

Specifier	Meaning
%c	Character
%s	String
%p	Pointer address
%d	Signed decimal integer
%i	Signed integer
%u	Unsigned decimal integer
%x	%x Lowercase hexadecimal
%X	Uppercase hexadecimal
%%	Percent character

> Example printf

```c
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    const char* s = "Hello";
    printf("Strings:\n"); // same as puts("Strings");
    printf(" padding:\n");
    printf("\t[%10s]\n", s);
    printf("\t[%-10s]\n", s);
    printf("\t[%*s]\n", 10, s);
    printf(" truncating:\n");
    printf("\t%.4s\n", s);
    printf("\t%.*s\n", 3, s);

    printf("Characters:\t%c %%\n", 'A');

    printf("Integers:\n");
    printf("\tDecimal:\t%i %d %.6i %i %.0i %+i %i\n",
                         1, 2,   3, 0,   0,  4,-4);
    printf("\tHexadecimal:\t%x %x %X %#x\n", 5, 10, 10, 6);
    printf("\tOctal:\t\t%o %#o %#o\n", 10, 10, 4);

    printf("Floating-point:\n");
    printf("\tRounding:\t%f %.0f %.32f\n", 1.5, 1.5, 1.3);
    printf("\tPadding:\t%05.2f %.2f %5.2f\n", 1.5, 1.5, 1.5);
    printf("\tScientific:\t%E %e\n", 1.5, 1.5);
    printf("\tHexadecimal:\t%a %A\n", 1.5, 1.5);
    printf("\tSpecial values:\t0/0=%g 1/0=%g\n", 0.0 / 0.0, 1.0 / 0.0);

    printf("Fixed-width types:\n");
    printf("\tLargest 32-bit value is %" PRIu32 " or %#" PRIx32 "\n",
                                     UINT32_MAX,     UINT32_MAX );
}
```

> Example Output

```
Strings:
 padding:
        [     Hello]
        [Hello     ]
```

```
        [     Hello]
 truncating:
        Hell
        Hel
Characters:     A %
Integers:
        Decimal:        1 2 000003 0  +4 -4
        Hexadecimal:    5 a A 0x6
        Octal:          12 012 04
Floating-point:
        Rounding:       1.500000 2 1.3000000000000000444089209850062 6
        Padding:        01.50 1.50  1.50
        Scientific:     1.500000E+00 1.500000e+00
        Hexadecimal:    0x1.8p+0 0X1.8P+0
        Special values: 0/0=-nan 1/0=inf
Fixed-width types:
        Largest 32-bit value is 4294967295 or 0xffffffff
```

Step 3: write() — Output and System Calls

1. What is a system call?
2. What is a file descriptor?
3. What are STDIN_FILENO , STDOUT_FILENO , and STDERR_FILENO ?
4. What does the return value of write() represent?
5. What is a partial write?
6. How can write() fail?
7. What is the difference between writing to a file and writing to a terminal?

Step 4: Number Representation — Decimal and Hexadecimal

Research questions

1. What is base 10?
2. What is base 16?
3. How do you convert a decimal integer to hexadecimal?
4. What is the difference between signed and unsigned integers?
5. What are INT_MIN , INT_MAX , and UINT_MAX ?
6. What is integer overflow?
7. What is the difference between int , unsigned int , long , and unsigned long ?
8. Why is INT_MIN a special case when converting a negative number to a positive magnitude?

C conversios[^4]

Step 5: Parsing Algorithms — Reading the Format String

Research questions

```
1. What is parsing?
2. What is string traversal?
3. How can a program distinguish normal characters from % ?
4. What should happen after the parser finds % ?
5. How can a parser identify a conversion specifier?
6. What is a finite state machine?
7. How can a switch statement help organize conversion handling?
```

Suggested topics to search

```
- String Parsing in C
- Finite State Machine in C
- Format String Parsing
- Character Classification in C
```

> Example of Parsing a Template String

```c
/* Test whether the nargs specified objects
   in the vector args are valid
   for the format string format:
   if so, return 1.
   If not, return 0 after printing an error message.  */

int
validate_args (char *format, int nargs, OBJECT *args)
{
  int *argtypes;
  int nwanted;

  /* Get the information about the arguments.
     Each conversion specification must be at least two characters
     long, so there cannot be more specifications than half the
     length of the string.  */

  argtypes = (int *) alloca (strlen (format) / 2 * sizeof (int));
```

```
  nwanted = parse_printf_format (format, nargs, argtypes);

  /* Check the number of arguments.  */
  if (nwanted > nargs)
    {
      error ("too few arguments (at least %d required)", nwanted);
      return 0;
    }

  /* Check the C type wanted for each argument
     and see if the object given is suitable.  */
  for (i = 0; i < nwanted; i++)
    {
      int wanted;

      if (argtypes[i] & PA_FLAG_PTR)
    wanted = STRUCTURE;
      else
    switch (argtypes[i] & ~PA_FLAG_MASK)
      {
      case PA_INT:
      case PA_FLOAT:
      case PA_DOUBLE:
        wanted = NUMBER;
        break;
      case PA_CHAR:
        wanted = CHAR;
        break;
      case PA_STRING:
        wanted = STRING;
        break;
      case PA_POINTER:
        wanted = STRUCTURE;
        break;
      }
      if (TYPE (args[i]) != wanted)
    {
      error ("type mismatch for arg number %d", i);
      return 0;
    }
    }
  return 1;
}
```

Step 6: `Function Pointers` and `Conversion Handlers`

Research questions

```
1. What is a function pointer?
2. How do you declare and call one?
3. What is an array of function pointers?
4. What is a dispatch table?
5. When does this approach make code easier to maintain?
6. Is it necessary for the mandatory part of ft_printf ?
```

Suggested topics to search

```
- Function Pointers in C
- Array of Function Pointers in C
- Dispatch Table in C
```

Step 7: Testing and Debugging

Testing checklist

Basic conversions

```
- [] %c
- [] %s
- [] %p
- [] %d
- [] %i
- [] %u
- [] %x
- [] %X
- [] %%
```

String edge cases

```
- [] Empty string
- [] Null string pointer
- [] Single-character string
- [] Long string
- [] Multiple string conversions
```

Integer edge cases

```
- [] 0
- [] -1
- [] INT_MAX
- [] INT_MIN
- [] UINT_MAX
- [] Large hexadecimal values
```

Format-string cases

```
- []  Empty format string
- []  Format string without conversions
- []  Consecutive `%%`
- []  Multiple conversions
- []  Trailing `%`
```

Return-value cases

```
- [] Only normal characters
- [] Only conversions
- [] Mixed normal characters and conversions
- [] Empty output
- [] Compare the return value with the original printf
```

Instructions

- tha command he compeal all file and he return libft.a

```
make
```

- This cleaning command removes all *.o files

```
make clean
```

- This depends on `clean`; once that has finished, it deletes `NAME` and everything else created by `make` or `make all`.

```
make fcean
```

- This command depends on other commands; it deletes everything that was created via `make` and then rebuilds it from scratch.

```
make re
```

- test main.c

```c
#include "ft_printf.h"

int main(void)
{
    ft_printf("Hello %s!\n", "42");
    ft_printf("Number: %d\n", 42);
    return (0);
}
```

```
cc main.c -L. -lftprintf -o test
```

Resources

> us AI

- I only used AI to explain a few simple concepts that I didn't fully understand. I tried to follow the old-school approach by doing my own research and looking up information on Google to learn things independently.

What are variadic functions in C?

The GNU C Library